DS 5030

10/30/26

Jack Brennan, Austin Blackburn, Austin Alvarado, Kyle Alt, Cameron Barlow

Lab 7 Report

1. Problem summary

In this lab, we are acting as a seller trying to figure out what price to charge without seeing the demand curve. Each time we set a price, a random number of customers enter the market and either buy or do not buy. As prices increase, the probability a customer buys decreases. Therefore, raising the price leads to more revenue per sale but also fewer actual sales. Since customer arrivals and purchases are random, one experiment cannot really tell us if a price change will actually increase revenue or not. For instance, increasing the price on one batch of customers may lead to more revenue, but on another batch sales decrease to the point where revenue is lost. On top of this, revenue per customer is not just a simple curve, as there are several prices in the 0-70 range that appear to be optiums, however only one is the true global optimal price, so any kind of search that just explores locally could completely miss the best price. Every customer that is a part of testing matters, so customers who are tested on with a bad price are revenues lost. Therefore, our goal was to build an algorithm to search for the best price on its own, keep track of how many customers are tested on, and capture as much of the optimal $11.925 profit per customer as possible across the entire experiment.

2. **Design and uncertainty**

To estimate how revenue changes with price, each round tests the prices, $p - 1$ and $p + 1$, on 1000 simulated customers each and takes the difference in mean revenue per customer divided by the price gap. We just try one price slightly higher and one slightly lower and see which earns more. Since we use a lot of customers each time, the luck of who shows up doesn't mess it up too much. The price then moves in the direction of the slope with a step size of $3/\sqrt{t}$, so early rounds move quickly and later rounds shrink and average out noise. Each move is also capped at plus or minus 2 dollars, so a single unlucky gradient estimate can't throw the price across the range.

The revenue curve has a few local peaks, so if we just climb once we might get stuck on the wrong one. We also only see noisy results from individual customers, so we can't tell ahead of time which peak is really the highest. To deal with this, we start the search from 14 prices spread evenly between 5 and 70 dollars, and each one climbs for 20 rounds. That gives us 14 endpoints. Since the climbs run on noisy data, an endpoint might end up a little off its peak, and endpoints that land close together might just be the same peak. So looking at where they land doesn't tell us which one is best. To pick one, we test every endpoint again on a new group of 20,000 customers. We use new customers so the test doesn't reuse the data that led us to those prices, and so a lucky run doesn't get an unfair advantage. Whichever price makes the most money per customer in that test gets picked. This test is still noisy, though. If two prices are really close, 20,000 customers might not be enough to tell them apart, so the winner is our best guess and not a sure thing.

3. **Challenges**

A challenge faced was the revenue curve has multiple peaks, so a climb can get stuck on a peak that is incorrect. Each climb only sees the slope right around its price, so it can't tell if there is a better peak somewhere else. In our results, climbs starting at $5, $10, or $15 all ended at the peak near $12, climbs from $20 to $35 ended near $27, and climbs from $50 to $65 ended near $52. With only a few starting prices, a search could settle at the $12 or $52 peaks and miss the ideal at $27. This matters because every future customer pays the price we settle on, so a worse peak loses money on each one. To combat this, we used 14 different starting prices across the range so that climbs land on each peak. This fixed the problem, but it meant testing more customers.

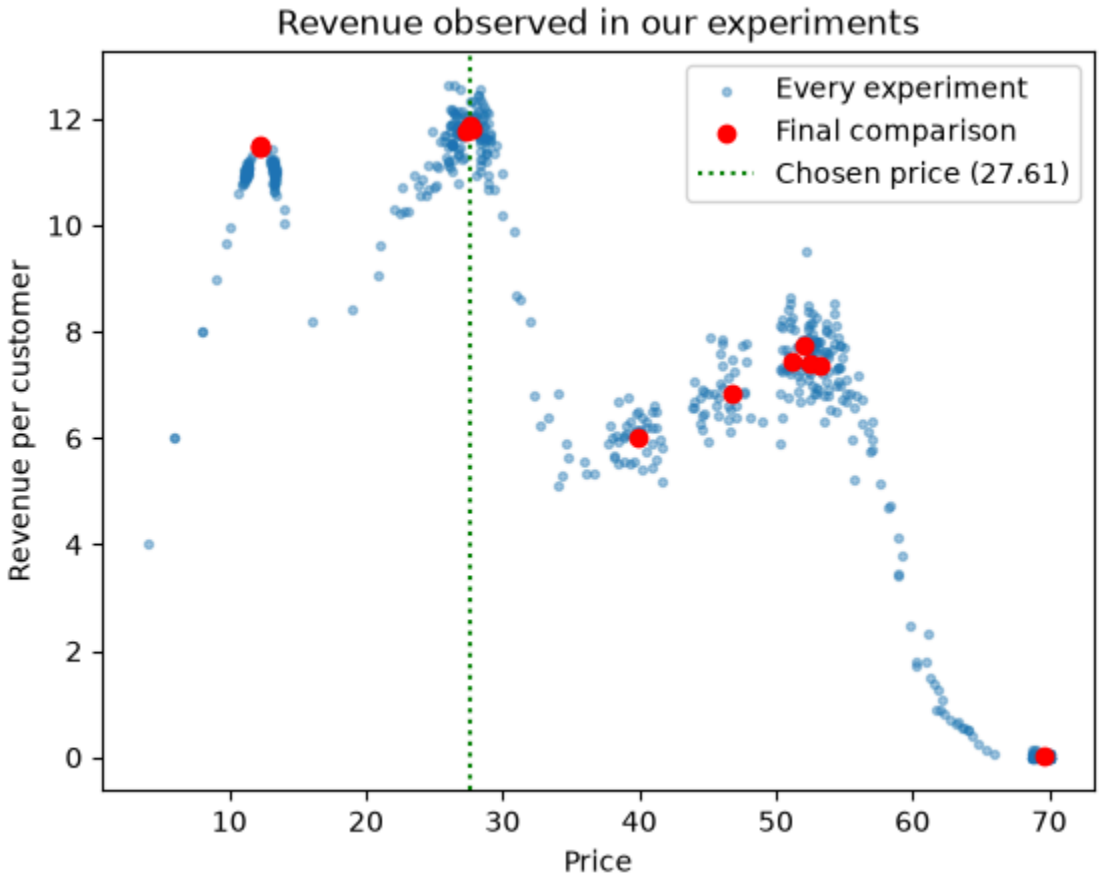
Another challenge was the cost of testing, and every customer we test is a customer we could have earned more from. With 14 starts, 20 rounds, 2 prices per round, and 1,000 customers per price, climbs used up about 560,000 customers, and the final comparison used another 280,000. A lot of those customers were tested at prices that were bad. For example, the climb that stayed at $70 earns about $0.02 per customer and still got 20,000 customers in the final comparison. We accepted this cost, but we could have cut it by stopping climbs that were going

nowhere early, using fewer customers at bad prices, and only running the final comparison on the best few endpoints.

4. Results

Figure 2

Plot of the revenue per customer by price



The experiments that we ran found that the optimal price to generate the most revenue per customer is $27.61. The total number of customers tested was 838,852, generating $7,078,116 in total revenue, meaning the average revenue per customer is $8.44. This is 70.8% of the desired $11.925 per customer benchmark. The two most successful price ranges were ~$12 and ~$27 as shown by the clusters around those prices in Figure 2. The revenue per customer around these clusters ranged around $11-12, with our experiments just missing the true optimal

price at $27.26. As price increases, revenue per customer generally decreases, with there being a brief positive trend between $40 - $51, before it sharply decreases until it reaches $70.